

Flash Attention-2

(Faster Attention with better Parallelism and work Partitioning)

Abstract:

- Scaling Transformers to longer Sequence length is a major problem but it promises to improve performance in language modeling & high resolution image understanding.
- Attention layer is main bottleneck in Scaling to longer Sequences, as its runtime and memory increase quadratically in Sequence length.
- Flash Attention-1 exploits the asymmetric GPU Memory hierarchy to bring ~~significant~~ significant memory saving (linear instead of quadratic) and runtime speedup (2-4x compared to optimized baselines) with no approximation.
- Flash Attention-1 is still not nearly as fast as optimized matrix multiply (GEMM) operations, reaching only 25-40% of theoretical maximum FLOPs/s. It is due to suboptimal work partitioning b/w different thread blocks & warps on the GPU, causing low occupancy (or unnecessary shared ~~read~~ memory reads/writes).
- Flash Attention-2 does better work Partitioning in parallel:
 - (1) Tweak the Algorithm to reduce num. of non-Matrix FLOPs
 - (2) Parallelize attention computation, even for a single head, across different thread blocks to increase occupancy
 - (3) Within each thread block, distribute the work b/w warps to reduce communication reads/writes through shared memory. These yield 2x speedup in comparison to Flash Attention-1, reach 50-73% theo. max FLOPs/100.

(1) Introduction

- Flash Atten. - 1 reordered computation using classical techniques like tiling, recomputation to reduce memory usage from quadratic to linear in sequence length.

(hence it was adopted in large scale training & Inference of Transformers)

→ cons. $\rightarrow$ 30-50% of theo. max. FLOPs/s in forward Pass
All on ALU GPU $\rightarrow$ 25-35% " " in backward Pass
GEMM reaches 80-90% of theo. max.

- Flash Atten. - 2 addresses the following challenges.

(1) Though non-matmul FLOPs only account small fraction of total FLOPs, they take longer to perform as GPUs have specialized units for matrix multiply, as a result matrix throughput can be 16x higher than non-matmul throughput

(2) ~~Both~~ Both Forward & Backward Pass were parallelized along the sequence length dimension in addition to batch number & number of head dimension.

This increases occupancy & GPU utilization
[As in case of sequences are long batch size is often small]

(3) [Nothing new to add refer prev page]

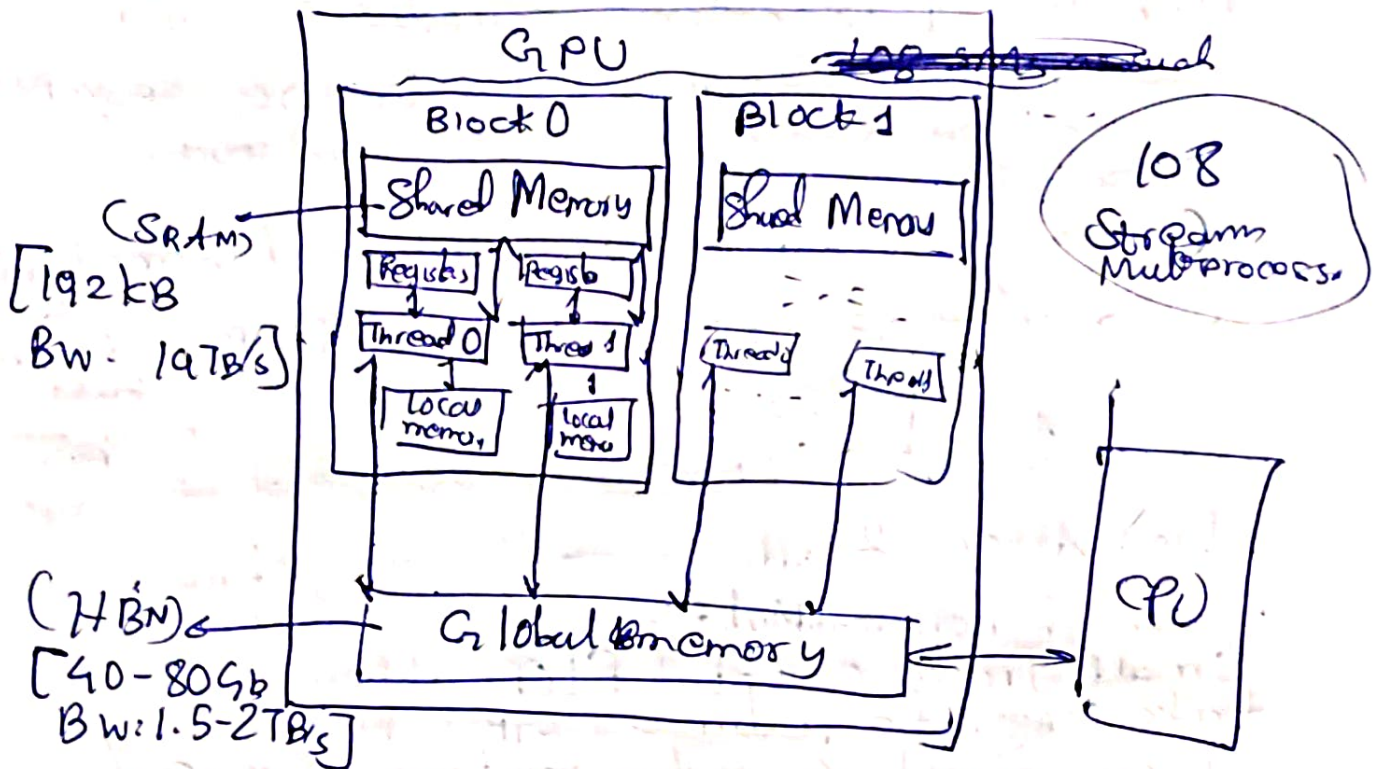
(2) Background

2.1 Hardware Characteristics

GPU Characteristics

- GPUs contain specialized units to accelerate matrix multiply in low precision (eg Tensor Cores on Nvidia)
- The memory hierarchy comprise of high bandwidth memory (HBM) and on chip SRAM (aka shared memory)
- L2 Cache is not directly controllable by user

Fig: Structure of GPU Memm



Execution Model

- GPUs have a massive no. of threads to execute an operation (called a kernel). Threads are organized into thread blocks which are scheduled to run on streaming multiprocessors (SMs). Within each

thread blocks, threads are grouped into warps (a group of 32 threads).

Threads within a warp can communicate by fast shuffle instructions

or cooperate to perform matrix multiply.

Warps within a thread block can communicate by reading from (or) writing to shared memory. Each kernel loads input from HBM to registers & SRAM, completes their execution and outputs to HBM.

$$\text{diag}(P) - PPT \text{ doVT}$$

$$\text{diag}(P) \text{ VT} - PPT \text{ VT}$$

$$0 \text{ do} \rightarrow P(P - 0 \text{ do})$$

2.2 Standard Attention Implementation

Recall $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Given input sequences $Q, K, V \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension
we want to compute the attention O, P
 $O \in \mathbb{R}^{N \times d}$

$S = QK^T \in \mathbb{R}^{N \times N}$, $P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$, $O = PV \in \mathbb{R}^{N \times d}$
where softmax is applied row-wise.

For multi-headed attention (MHA), this same computation is performed in parallel across many heads, and parallel on the batch ~~dimension~~ dimension.

The backward pass of attention proceeds as follows:
Let $dO \in \mathbb{R}^{N \times d}$ be the gradient of O w.r.t. respect to some loss function. Then by the chain rule (aka backpropagation):

$$dV = P^T dO \in \mathbb{R}^{N \times d} \quad dP = dO V^T \in \mathbb{R}^{N \times N}$$

$$dS = d\text{softmax}(dP) \in \mathbb{R}^{N \times N} \quad dQ = dS K \in \mathbb{R}^{N \times d}$$

$$dK = dS^T Q \in \mathbb{R}^{N \times d}$$

where $d\text{softmax}$ is the gradient (backward pass) of softmax applied row-wise.

[Note: if $P = \text{softmax}(S)$ for some vector S and P , then with w.r.t. gradient dP , the w.r.t. gradient $dS = (\text{diag}(P) - PP^T) dP$ (proven)]

- Standard attention implementations materialize the matrices S and P to HBM, which takes $\mathcal{O}(N^2)$ memory. Often $N \gg d$ (typical N is on the order of 1k-8k and d is around 64-128). The standard attention implementation (1) calls the matrix multiply (GEMM) subroutine to multiply $S = QK^T$ writes the result to HBM.

(2) load S from HBM to compute softmax
compute the result P to HBM to
HBM.

(3) calls GEMM to get $O = PV$

As most of operations are bounded by memory
bandwidth, the large number of accesses
translates to slow wall clock time.

Moreover the required memory is $O(N^2)$
due to having to materialize S and P .

Moreover one has to store $P \in \mathbb{R}^{N \times N}$ for
the backpass to compute the grads

2.3 Flashattention-1

2.3.1 Forward Pass

Flash Attention-1 applies the classical technique of
tiling to reduce memory IOs by

(1) loading blocks of inputs from HBM to SRAM

(2) computing attention by colb-wise
to that block and then

(3) updating the output without
copying the large intermediate matrices
 S and P to HBM

As softmax computes entire rows or blocks of
rows, online softmax can split the
attention computation into blocks and
rescale the O/P of each block to
finally get the right result.

We describe the online softmax technique and how it is used in attention

For simplicity consider just one row block of attention matrix S of the

form $[S^{(1)} S^{(2)}]$ for some matrices $S^{(1)}, S^{(2)} \in \mathbb{R}^{B_r \times B_c}$ where B_r and B_c are row and column block sizes. We want to compute softmax of this row block and multiply with the value, of the form $\begin{bmatrix} V^{(1)} \\ V^{(2)} \end{bmatrix}$ for some matrices $V^{(1)}, V^{(2)} \in \mathbb{R}^{B_c \times d}$

Standard softmax would compute

$$m = \max(\text{rowmax}(S^{(1)}), \text{rowmax}(S^{(2)})) \in \mathbb{R}^{B_r}$$

$$l = \text{rowsum}(e^{S^{(1)} - m}) + \text{rowsum}(e^{S^{(2)} - m}) \in \mathbb{R}^{B_r}$$

$$p = \begin{bmatrix} p^{(1)} & p^{(2)} \end{bmatrix} = (\text{diag}(l))^{-1} \begin{bmatrix} e^{S^{(1)} - m} & e^{S^{(2)} - m} \end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c}$$

$$O = \begin{bmatrix} p^{(1)} & p^{(2)} \end{bmatrix} \begin{bmatrix} V^{(1)} \\ V^{(2)} \end{bmatrix}$$

$$O = (\text{diag}(l))^{-1} (e^{S^{(1)} - m} V^{(1)} + e^{S^{(2)} - m} V^{(2)}) \in \mathbb{R}^{B_r \times d}$$

Online softmax instead computes local softmax with respect to each block & rescale to get the right output at the end

$$m^{(1)} = \text{rowmax}(S^{(1)}) \in \mathbb{R}^{B_0}$$

$$l^{(1)} = \text{rowsum}(e^{S^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_0}$$

$$\hat{p}^{(1)} = \text{diag}(l^{(1)})^{-1} e^{S^{(1)} - m^{(1)}} \in \mathbb{R}^{B_0 \times B_0}$$

$$O^{(1)} = \hat{p}^{(1)} V^{(1)} = \text{diag}(l^{(1)})^{-1} e^{S^{(1)} - m^{(1)}} V^{(1)} \in \mathbb{R}^{B_0 \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(S^{(2)})) = m$$

$$l^{(2)} = (e^{m^{(1)} - m^{(2)}}) l^{(1)} + \text{rowsum}(e^{S^{(2)} - m^{(2)}})$$

$$= \text{rowsum}(e^{S^{(1)} - m}) + \text{rowsum}(e^{S^{(2)} - m})$$

$$= l$$

$$\hat{p}^{(2)} = \text{diag}(l^{(2)})^{-1} e^{S^{(2)} - m^{(2)}}$$

$$O^{(2)} = \text{diag}\left(\frac{l^{(1)}}{l^{(2)}}\right) O^{(1)} + \hat{p}^{(2)} V^{(2)}$$

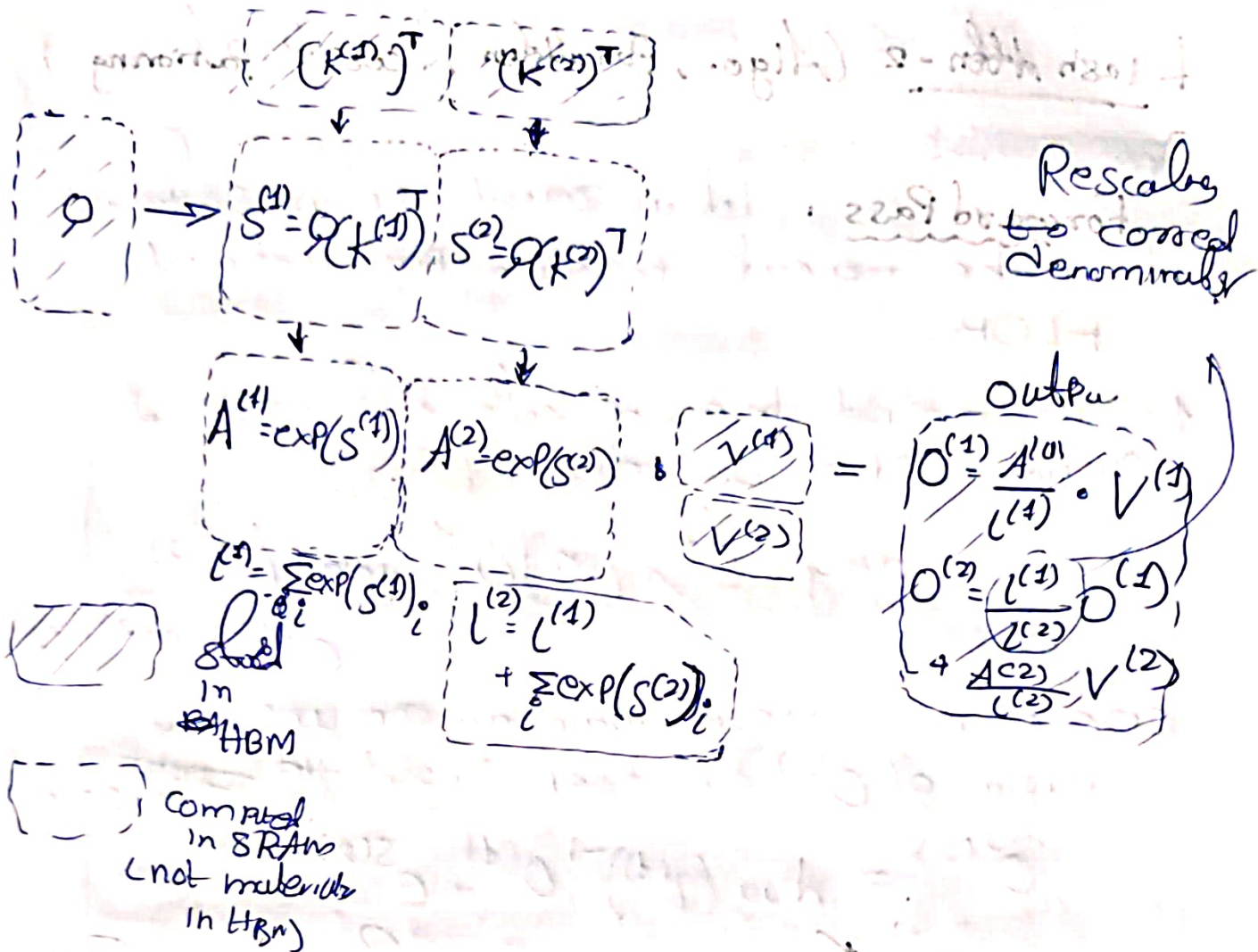
$$= \text{diag}(l^{(2)})^{-1} e^{S^{(1)} - m} V^{(1)} + \text{diag}(l^{(2)})^{-1} e^{S^{(2)} - m} V^{(2)}$$

$$= O$$

post this
they are as sum
 $l^{(1)} \leftarrow l^{(1)} e^{m^{(1)} - m^{(2)}}$

you can
seq
to this
simplified.

we show how ~~online~~ flash Attention-1
uses online software to enable tiling
to reduce reads/writes



2.3.2. Backward Pass

In the backward pass, by recomputing the attention matrices S and P once blocks of inputs Q, K, V are already loaded to SRAM, Flash-attention-1 avoids having to store large intermediate values. By not having to save the large matrices S and P of size $N \times N$, Flash Attn-1 yields 10-20x memory savings. The backward pass also achieves a 2-4x wall clock speedup due to reduced memory reads/writes.

The backward pass applies tiling to eqn in Sec. 2.2. Though it is simpler than forward pass (there is no softmax rescale), the implementation is involved because more values to be kept in SRAM to perform 5 matrix mult. (compared to 3 in forward).

③ Flash Attention - 2 (Algo., Parallelism & Cook Partitioning)

~~We describe~~

Forward Pass: let us consult online resources & make tweaks to reduce non-matrix FLOPs.

1) we do not have to scale both terms & output is updated by $\text{diag}(l^{(2)})^{-1}$

$$O^{(2)} = \text{diag}(l^{(1)} / l^{(2)}) O^{(1)} + \text{diag}(l^{(2)})^{-1} e^{sc(2) - m^{(2)}} \sqrt{v^{(2)}}$$

we can instead maintain an unscaled version of $O^{(2)}$ and keep around the ~~stats~~ Statistics ⁽²⁾

$$\tilde{O}^{(2)} = \text{diag}(l^{(1)})^{-1} O^{(1)} + e^{sc(2) - m^{(2)}} \sqrt{v^{(2)}}$$

[I do not think inverse should appear here. ?? rather $(l^{(1)})' \neq e^{m^{(2)} m^{(1)}} (l^{(1)})$ $\text{diag}(l^{(1)})' O^{(1)} + e^{sc(2) - m^{(2)}} \sqrt{v^{(2)}}$ $\stackrel{\leq m^{(2)} m^{(1)}}{=} \tilde{O}^{(1)} + e^{sc(2) - m^{(2)}} \sqrt{v^{(2)}}$]

only at the very end of loop do we scale the $\tilde{O}^{(last)}$ by $\text{diag}(l^{(last)})^{-1}$ to get the right O

2) we do not have to save both the m & $m^{(i)}$ and sum of ~~exponent~~ (i) for backward pass we only need to ~~save~~ store $\log \text{sumexp } L^{(i)} = m^{(i)} + \log(l^{(i)})$

In the simple case a 2 block in the online softmax becho.

$$m^{(1)} = \text{rowmax}(S^{(1)}) \in \mathbb{R}^{B_0}$$

$$l^{(1)} = \text{rowsum}(e^{S^{(1)}} - m^{(1)}) \in \mathbb{R}^{B_1}$$

$$\tilde{O}^{(1)} = e^{S^{(1)} - m^{(1)}} V^{(1)} \in \mathbb{R}^{B_0 \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(S^{(2)})) = m$$

$$\begin{aligned} l^{(2)} &= e^{m^{(1)} - m^{(2)}} l^{(1)} + \text{rowsum}(e^{S^{(2)} - m^{(2)}}) \\ &= \text{rowsum}(e^{S^{(1)} - m}) + \text{rowsum}(e^{S^{(2)} - m}) \\ &= l \end{aligned}$$

$$\tilde{P}^{(2)} = \text{diag}(l^{(2)})^{-1} e^{S^{(2)} - m^{(2)}} \rightarrow \text{you can compute this}$$

$$\begin{aligned} \tilde{O}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}}) \tilde{O}^{(1)} + e^{S^{(2)} - m^{(2)}} V^{(2)} \\ &= e^{S^{(1)} - m} V^{(1)} + e^{S^{(2)} - m} V^{(2)} \end{aligned}$$

$$O^{(2)} = \text{diag}(l^{(2)})^{-1} \tilde{O}^{(2)} = O$$

Flash-2 Forward Pass

Recurse: Matrices $Q, K, V \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_r, B_c

(1) Divide Q into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $Q_1, \dots, Q_{T_r}$ of size $B_r \times d$ each, and divide K, V into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $K_1, K_2, \dots, K_{T_c}$ and $V_1, V_2, \dots, V_{T_c}$ of size $B_c \times d$ each

(2) Divide the output $O \in \mathbb{R}^{N \times d}$ into T_r blocks $O_1, \dots, O_{T_r}$ of size $B_r \times d$ each and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each

(3) for $1 \leq l \leq T_r$ do

Load Q_l from HBM to on-chip SRAM
on chip initialize $Q^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$
 $L^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$

for $1 \leq j \leq T_c$ do

load K_j, V_j from HBM to on-chip SRAM
on chip compute $S_i^{(j)} = Q_i \cdot K_j^T \in \mathbb{R}^{B_r \times B_r}$
on chip compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{softmax}(S_i^{(j)})) \in \mathbb{R}^{B_r}$
 $\tilde{p}_i^{(j)} = \exp(S_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_r}$

$$L_i^{(j)} = \exp(m_i^{(j-1)} - m_i^{(j)}) L_i^{(j-1)} + \text{softmax}(\tilde{p}_i^{(j)})$$

on chip compute $O_i^{(j)} = \text{diag}(\exp(m_i^{(j-1)} - m_i^{(j)})) O_i^{(j-1)}$

endfor

On chip, compute $O_i = \text{diag}(L_i^{(T_c)})^{-1} \frac{\tilde{p}_i^{(j)} V_j}{O_i^{(T_c)}}$

on chip, compute $L_i = m_i^{(T_c)} + \log(L_i^{(T_c)})$

Write O_i to HBM as i^{th} block of O

write L_i to HBM as the i^{th} block of L

endfor

Return the output O and the logsumexp L

Causal Masking

One common use case of attention is in auto regressive language modeling, where we need to apply a causal mask to the attention matrix S (i.e any entry S_{ij} with $j > i$ is set to $-\infty$)

1) As 'Flash Attn-1 & 2' already operated by blocks. for any blocks where all column indices are more than the row indices (approximately half of the blocks in large sequence length) we can skip the computation of that block.

This leads to 1.7-1.8x speedup compared to attention without the causal mask.

2) we do not need to apply the causal mask for blocks whose row indices are strictly less than column indices. This means that for each row, we only need to apply causal mask to 1 block (assuming square block)

0: ~~The cost is~~ $O(N^2 d)$ FLOPs
This requires $O(N)$ additional memory beyond inputs and output.

(Proof was said to be similar FlashAttn-1)

Backward Pass

The backward pass of FlashAttn-2 is almost same as ~~as~~ that of FlashAttn-1

We make a minor tweak to only use the row-wise LogSumExp instead of both the row-wise max and row-wise sum of exponentials in the Softmax

FlashAttn-2 Backward Pass

Require: Matrices $Q, K, V, O, d \in \mathbb{R}^{N \times d}$ in HBM
vector $L \in \mathbb{R}^N$, block sizes B_r, B_c

1) Divide Q into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $Q_1, \dots, Q_{T_r}$ of size $B_r \times d$ each, and ~~and~~ divide K, V into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $K_1, \dots, K_{T_c}$ and $V_1, \dots, V_{T_c}$ of size $B_c \times d$ size

2) Divide O into T_r blocks $O_1, \dots, O_{T_r}$ of size $B_r \times d$ each, divide dO into T_r blocks $dO_1, \dots, dO_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each

3) Initialize $dO = (0)_{N \times d}$ in HBM and divide it into T_r blocks $dO_1, \dots, dO_{T_r}$ of size $B_r \times d$ each. Divide $dK, dV \in \mathbb{R}^{N \times d}$ into T_c blocks $dK_1, \dots, dK_{T_c}$ and $dV_1, \dots, dV_{T_c}$ of size $B_c \times d$ each.

4) Compute $D = \text{rowSum}(dO \cdot O) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.

5) for $1 \leq j \leq T_c$ do
Load K_j, V_j from HBM to on chip SRAM

① Initialize $dK_j = (0)_{B_c \times d}, dV_j = (0)_{B_c \times d}$ on SRAM

for $1 \leq i \leq T_r$ do

Recall we have $L_i = m_i^{(T_c)} + \log(l_i^{(T_c)})$
so $\exp(S_{ij} - m_i^{(T_c)} - \log(l_i^{(T_c)}))$
so $\frac{\exp(S_{ij} - m_i^{(T_c)})}{(l_i)^{T_c} \max_{1 \leq k \leq T_c} \{l_k\}}$
Total sum of den. term that i th block

Load $O_i, O_i^T, dO_i, dO_i^T, L_i, D_i$ from HBM to on chip SRAM
On chip, compute $S_i^{(j)} = O_i \cdot K_j^T \in \mathbb{R}^{B_r \times B_c}$
On chip, compute $P_i^{(j)} = \exp(S_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$
on chip compute $dV_j \leftarrow dV_j + (P_i^{(j)})^T dO_i$
On chip compute $dP_i^{(j)} \leftarrow dO_i \cdot V_j^T \in \mathbb{R}^{B_r \times B_c}$
On chip compute $dS_i^{(j)} = P_i^{(j)} \odot (dP_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times d}$

push $dS_i^{(j)}$ to end answer

Why only dO_i in HBM while dK_j, dV_j in SRAM?
①-2

Load dO_i from HBM to SRAM then on chip update $dO_i \leftarrow dO_i + dS_i^{(j)} K_j$ and write back to HBM $\in \mathbb{R}^{B_r \times d}$

①-2 on chip compute $dK_j \leftarrow dK_j + dS_i^{(j)} O_i \in \mathbb{R}^{B_c \times d}$
end for
write dK_j, dV_j to HBM
end for
return dO, dK, dV

o Multi Query attention (MQA) and grouped query attention (GQA) are variants of attention where multiple heads of query attend to same head of key and value in order to reduce the size of KV-cache during inference. Instead of having to duplicate the key and value heads for the computation, we implicitly manipulate the indices into head to perform the same computation. In the backward pass we need to sum the gradients dK and dV across different heads that were implicitly duplicated.

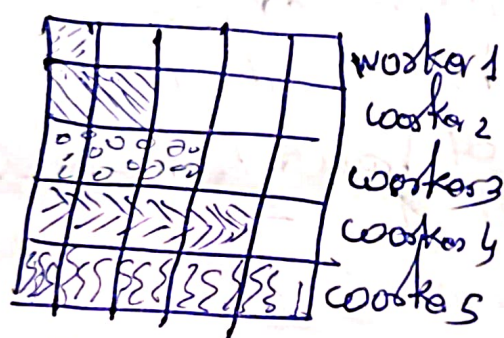
Parallelism

The first version of flashAttn parallelizes over batch size and number of heads. We use 1 threaded block to process one head, and there are overall $\text{batch size} * \text{number of heads}$ threaded blocks. Each threaded block is scheduled to run on a Streaming Multiprocessor (SM) and there are 108 SMs on A100. This scheduling is efficient when number of cores $(S_{\text{sm}} \geq 8)$. Since we can effectively use almost all of compute resource on the GPU.

- In the case of long sequences (which usually means small batch sizes or small number of heads) to make better use of multiprocessor on the GPU, we can additionally parallelize over the sequence length dimension. This results in significant speedup for this regime.
 - Forward Pass: we see that (over sequence length) is embarrassingly parallel, and we schedule them on different thread blocks that do not need to communicate with each other. We also parallelize over the batch dimension and number of head dimension as done in Flash Attn-1. The increased parallelism over sequence length helps improve utilization of unutilized GPU resource being used when the batch size and number of heads are small. Leads to speedup in this case.
- [Does parallelizing over sequence length not affect accuracy? mean some part of context might be prov. present how are they independent?]

Backward Pass: Notice that only shared computations b/w different column blocks is in $update dQ$ in Algorithm where we need to load dQ_i from HBM to SRAM, then on chip update $dQ_i \leftarrow dQ_i + dS_i^{(p)} K_j$ and write back to HBM. We thus parallelize over the Sequence length dimension as well, and schedule 1 thread block for each column block of backward pass. We use atomic adds to communicate b/w different thread blocks to update dQ .

We describe the parallelization scheme



Forward Pass



Backward Pass

$(dQ_i \leftarrow dQ_i + dS_i^{(p)} K_j)$
makes other workers to wait for prev. one

(In the forward pass (left); we parallelize the workers (three blocks where each worker takes care of a block of rows of the attention matrix. In the backward pass (right); each takes care of block of columns of attention matrix)

Decoding: During LLM inference, most of the time is

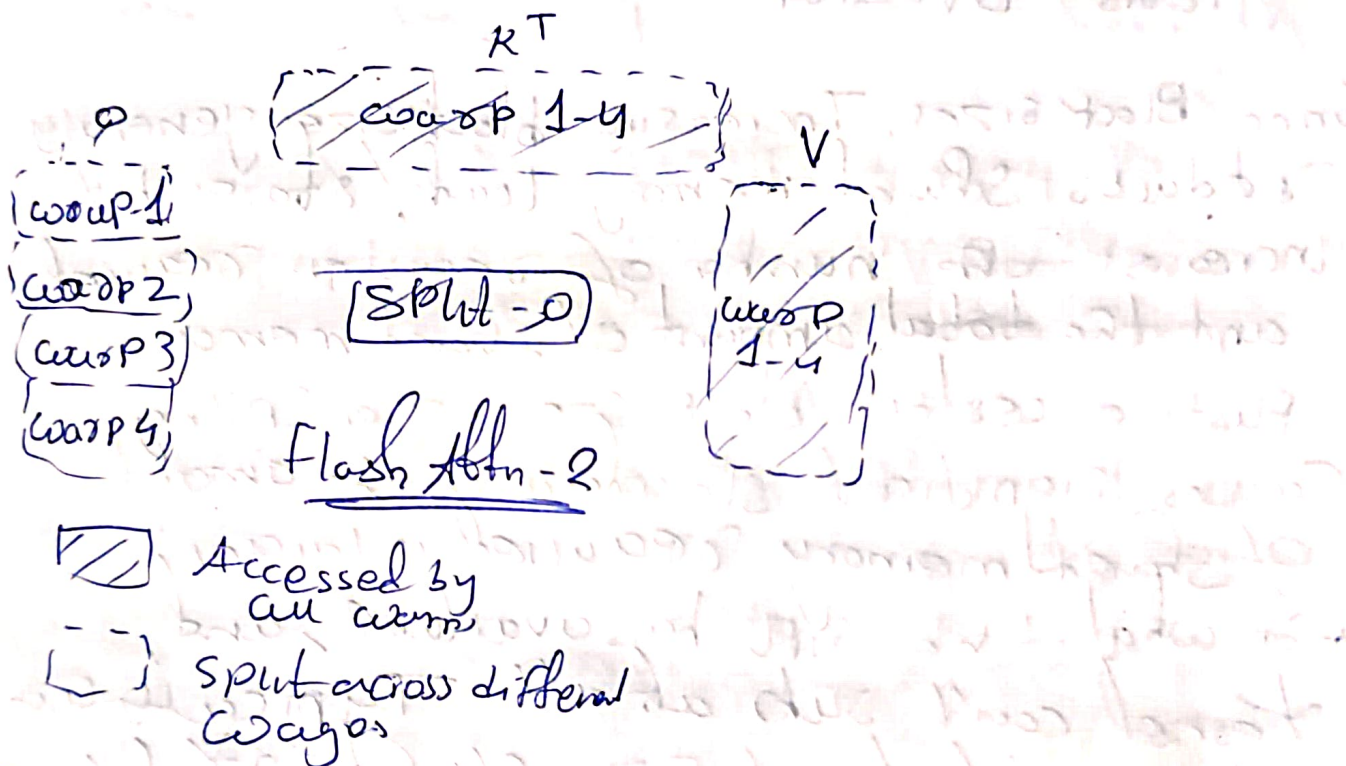
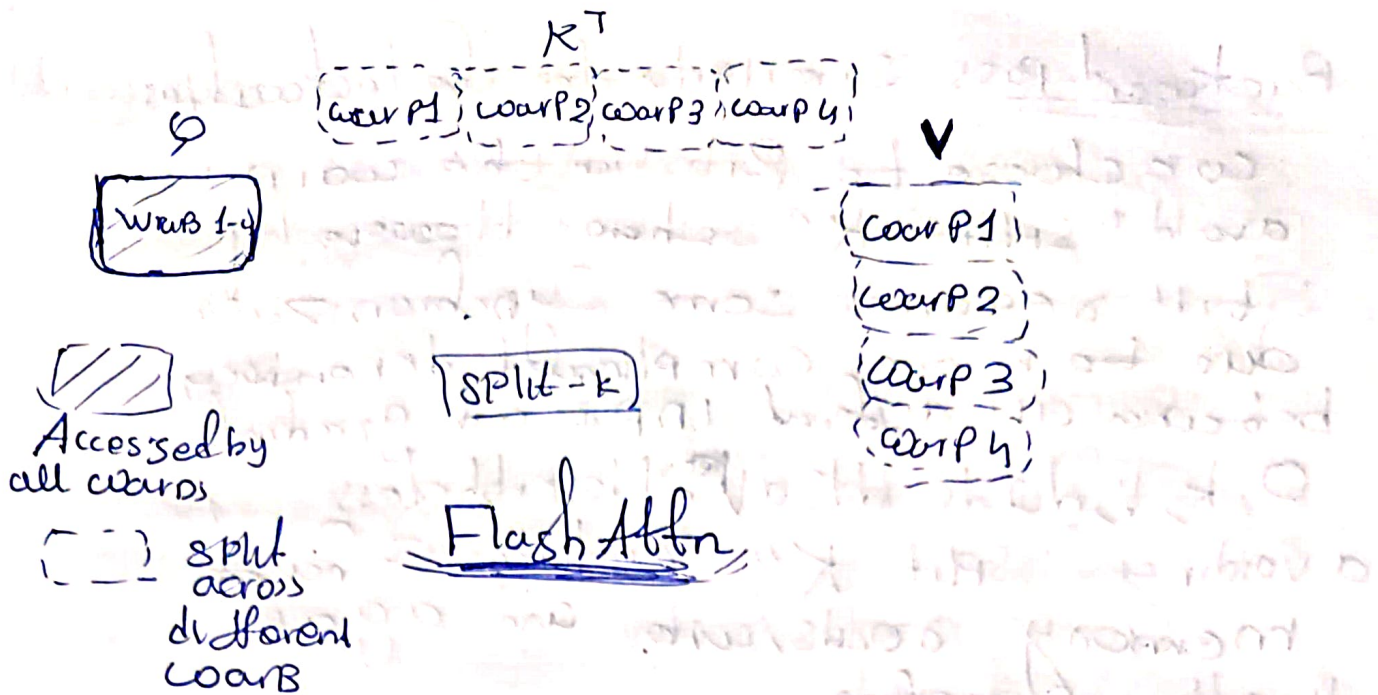
Spent on iterative decoding, where one token is predicted at a time. The bottleneck for attention operations during decoding is different from that during training or Prefill (prompt processing) because the query length is very short (often query length is 1 since only the new extra token is adding, all previous tokens stored in KV cache). As a result, the bottleneck is no longer the deadweight of intermediate matrices the score of $1 \times T$ and attention probabilities softmax ($1 \times T$). Instead, the bottleneck is load the KV cache as quickly as possible.

To accommodate this softmax, we split the KV cache loads among different threads/blocks, to increase occupancy and saturate the HBM bandwidth. However, since the thread blocks can't easily communicate with each other, we write intermediate results to HBM, then call a separate kernel to reduce the results and produce final O/P.

Work Partitioning between COGAs

As ~~see~~ in previous section we described how to schedule thread blocks, even within each thread block, we also have to decide how to partition the work between different warps. We typically use 4 or 8 warps per thread block, and the partitioning is described in following fig.

Forward Pass For each block FlashAtt-1 splits K and V across 4 warps while keeping Q accessible by all warps. Each warp multiplies to get a slice of QK^T then they need to multiply with a slice of V and communicate to add up the result. This is referred as the SPLIT & share. However this is inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then all warps read those shared memory results/writes slow down the forward pass in flashattn-1. In flashattn-2 we instead split Q across 4 warps while keeping K & V accessible by all warps. After each warp performs matrix multiply to get a slice of QK^T they just need to multiply their shared slice of V to get corresponding slice of o/p . There is no need ~~for~~ for communication b/w warps. The reduction in shared memory reads/writes yields speedup.



To understand why split-0 will require ~~less~~ no intermediate writes to shared mem. with split-k requires we need to see softmax+weighted sum require a reduction over the whole key dimension (i.e. entire row). In split-k layout the keys/values are partitioned across warps so each warp only sees a a slice (or a part of row) of the reduction hence it must write partial results. With the split-0 layout each warp owns a disjoint set of queries and has access to full K and V so every warp can compute full softmax normalized and final weighted sum so it is a **local** operation. No inter-card writes/reduces is needed.

Backward-Pass Similarly for the backward pass, we choose to partition the warps to avoid "split-K" scheme. However it still requires some synchronization due to more complicated dependency between all different inputs and gradients $\partial_i, k, v, d_i, d_o, dk, dv$. Nevertheless avoiding "split-K" reduces shared memory reads/writes and again yields speedup.

• Tuning Block sizes. Increasing block sizes generally reduces shared memory loads/stores but increases the number of registers required and the total amount of shared memory. Past a certain block size, register spilling causes significant slowdown as the amount of shared memory required is larger than what the GPU has available, and kernel can't run at all. Typically we choose blocks of size $\{64, 128\}$ depending on the head dimension d_h and the device shared memory size. We manually tune for each head dimension since there are essentially only 4 block sizes. It can be automated in future work.

4.2 Empirical Validation

Benchmarking Attention for training

FLOPs in forward pass

$$= 4 \cdot \text{Seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}$$

With a causal mask, we need to divide this number by 2 to account for the fact that approximately half of entries are calculated. To get FLOPs of backward pass we multiply the forward pass FLOPs by 2.5 (since there are 2 matrices in forward pass & 5 matrices in the backward pass due to recomputation)

Benchmarking Attention for Inference

It is significantly faster (28x) from GPT Transformer. This is thanks to better cooK partitions where multiple thread blocks are loading the KV cache at the same time to saturate HBM Bandwidth.

END TO END Performance

Note that we calculated the flops by formula following Megaburn-LM

$$= 6 \cdot \text{Seqlen} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{Seqlen}^2$$

The first term accounts for FLOPs due to weight input multiplication and second term accounts for FLOPs due to attention

However one can argue second term
 should be treated as with causal mask
 we only need to compute approximately
 half no. of elements in att. We choose
 to follow the formula in the literature
 without diving attention FLOPs by 2 for
consistency

Additional to understand Background Pass
 Notes

we
 already
 know

$$dV = P^T dO \in \mathbb{R}^{N \times d}$$

$$dP = dO V^T \in \mathbb{R}^{N \times N}$$

$$dS = d \text{softmax}(dP) \in \mathbb{R}^{N \times N}$$

$$dQ = dS \cdot K \in \mathbb{R}^{N \times d}$$

$$dK = dS^T Q \in \mathbb{R}^{N \times d}$$

It is most imp

$$P = \text{softmax}(S) \text{ "rowwise"}$$

So the softmax Jacobian (for a fixed row i)
 gives

$$dS_i = (\text{diag}(P_i) - P_i P_i^T) dP_i$$

where This can be written element wise

~~$$dS_{ij} = P_{ij} - \sum_t P_{it} dP_{it}$$~~

$$dS_{ij} = P_{ij} dP_{jj} - \left(\sum_t P_{it} dP_{it} \right) P_{ij}$$

$$C_i = \sum_t P_{it} dP_{it}$$

$$[dS_i[1] \ dS_i[2] \ \dots \ dS_i[C_w]]$$

$$= \begin{bmatrix} P_i[1] & P_i[2] & \dots & P_i[C_w] \end{bmatrix}$$

~~$$= \begin{bmatrix} P_i[1] & P_i[2] & \dots & P_i[C_w] \end{bmatrix}^T$$~~

$$= \begin{bmatrix} P_i[1] & P_i[2] & \dots & P_i[C_w] \end{bmatrix} \begin{bmatrix} P_i[1] & P_i[2] & \dots & P_i[C_w] \end{bmatrix}$$

$$dS_{i1} = P_{i1} (dP_{i1} - C_i)$$

Now we have

$$C_i = \sum_j P_{ij} dP_{ij}$$

(we have $dP = dO \cdot V^T$)

$$dP_{ij} = dO_i \cdot V_j$$

V_j is j^{th} row of V

$$1 \leq i, j \leq n$$

$$C_i = \sum_j P_{ij} (dO_i \cdot V_j) \quad (\text{dot product } (scalar \cdot P_{ij}))$$

$$= dO_i \cdot (\sum_j P_{ij} V_j)$$

~~dot row~~

$$= dO_i \cdot O_i$$

$$= \sum_{k=1}^n dO_{ik} O_{ik}$$

we know $O = P \cdot V$

$n \times n \quad n \times n \quad n \times n$

$$O_{ij} = \sum_t P_{it} V_{tj}$$

$$O_i = \left[\sum_t P_{it} V_{t1}, \sum_t P_{it} V_{t2}, \dots, \sum_t P_{it} V_{tn} \right]$$

$$= \sum_t \begin{bmatrix} P_{it} V_{t1} & P_{it} V_{t2} & \dots & P_{it} V_{tn} \end{bmatrix}$$

$$= \sum_t P_{it} [V_{t1}, V_{t2}, \dots, V_{tn}]$$

$$= \sum_t P_{it} V_t$$

$$= D_i$$

where $D = \text{rowsum}(dO \cdot O)$

here $D_i = \text{rowsum}(dO_i \cdot O_i)$

$$\begin{bmatrix} P_i[1] \\ P_i[2] \\ \vdots \\ P_i[n] \end{bmatrix}$$

$$= \begin{bmatrix} P_i[1]^2 & P_i[1]P_i[2] & \dots & P_i[1]P_i[n] \\ P_i[1]P_i[2] & P_i[2]^2 & \dots & \dots \\ \vdots & \vdots & \ddots & \vdots \\ P_i[1]P_i[n] & \dots & \dots & P_i[n]^2 \end{bmatrix}$$

$$\begin{bmatrix} dP_i[1] \\ dP_i[2] \\ \vdots \\ dP_i[n] \end{bmatrix}$$

$$\begin{bmatrix} P_i[1] - P_i[1]^2 & -P_i[1]P_i[2] & \dots & -P_i[1]P_i[n] \\ -P_i[1]P_i[2] & \dots & \dots & \dots \\ \vdots & \vdots & \ddots & \vdots \\ -P_i[1]P_i[n] & \dots & \dots & \dots \end{bmatrix}$$

$$\begin{bmatrix} dP_i[1] \\ dP_i[2] \\ \vdots \\ dP_i[n] \end{bmatrix}$$

$$\begin{bmatrix} (P_i[1] - P_i[1]^2) dP_i[1] \\ + (-P_i[1]P_i[2]) dP_i[2] \\ + \dots \\ + (-P_i[1]P_i[n]) dP_i[n] \end{bmatrix}$$

$$dS_{i1} = P_i[1] dP_i[1]$$

$$- P_i[1] (P_i[1] dP_i[1] + P_i[2] dP_i[2] + \dots)$$

$$dS_{i1} = P_i[1] dP_i[1] - P_i[1] \sum_t P_i[t] dP_i[t]$$

$$dS_{ij} = P_{ij} (dP_{ij} - D_{ij})$$

$$dS_{ij} = P_{ij} dP_{ij} - P_{ij} D_{ij}$$

$$dS_i = [P_{i1} dP_{i1} - P_{i1} D_{i1}, P_{i2} dP_{i2} - P_{i2} D_{i2}, \dots]$$

$$= [P_{i1} dP_{i1}, P_{i2} dP_{i2}, \dots] - \underbrace{[P_{i1}, P_{i2}, \dots]}_{P_i} D_i$$

$$= P_i \circ dP_i - P_i D_i$$

← elements
P_{2,1}

←

$$\boxed{dS = P \circ dP - (D \cdot 1^T) \circ P}$$

$$dS_i = P_{i \cdot} (dP_{i \cdot} - D_{i \cdot})$$

$$\begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_n \end{bmatrix}$$

$$[1 \dots 1]$$

So final

$$dV = P^T dO$$

$$dP = dO \cdot V^T$$

$$dS = P \circ dP - (D \cdot 1^T) \circ P$$

$$= P \circ (dP - D \cdot 1^T)$$

← normal matrix

$$dP > (dS) \cdot K$$

$$dK = dS^T \cdot P$$

← slightly overloaded notation scalar multiple using 'o' elementwise multiply

$$\begin{bmatrix} P_1 & D_1 & \dots & D_1 \\ P_2 & D_2 & \dots & D_2 \\ \vdots & \vdots & \ddots & \vdots \\ P_n & D_n & \dots & D_n \end{bmatrix}$$

$$\begin{bmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{bmatrix}$$

$$D = \text{rowsum}(dO \cdot O)$$

(o → elementwise product)

Let us write in low code form

Pues & See if ~~you~~ we are doing sequentially this algorithm
It does not matter
you are working to chip to ~~SRAM~~ ~~DRAM~~
the point is when you

will ~~you be~~ ^{do it} parallelly by Parallelizing
~~outer~~ loops ~~i~~ then multiply
different blocks (with different i
will be updating same $d[i]$ (ie i)
thread worker assigned)

[~~Since there is dependency $d[i]$ by $i=1$~~
~~before $d[i]$ by $i=2$~~ [I don't think ~~there is dependency~~]]

So they need to write $\leftarrow$
to same memory location across
blocks ie HBM (SRAM is local
to block)

Problems [Tried to frame from this paper]

- Q1) ~~The~~ State the three high level changes that Flash Attention-2 paper introduces to address bottlenecks/inefficiency.
- Q2) Compare Split-K v/s split-Q w/o atomicity. Explain both. Why Flash Attention-2 prefers split-Q and describe a scenario where split-K might be preferable.
- Q3) In Backward Pass of Flash Attention-2 why dQ gets ~~updated~~ overwritten to HBM at each iteration while dK, dV are updated at SRAM (Shared memory) before writing back the cumulative result to HBM at end of iteration.
- Q4) Explain the specific algorithmic modification made in Flash Attention-2's online softmax technique compared to original Flash attention. Why does this modification ~~ensure~~ guarantee a performance improvement on GPU?

[Q1) Why is ~~causal masking~~ applying a causal mask gives a speedup of nearly 10x speed up [In Flash Attention-2] even though mask needs to be applied to entire attention matrix "S" before processing it further]

Not good

- Q5) Analyse the different parallelization strategies used in Flash Attention-2's forward & backward passes. Why does the backward pass require atomic operations for updating dQ, while the forward pass does not - need inter thread block communication? How does this design choice address occlusion problem?

with long sequences both small
batch sizes

Q> If flash attention-2 achieves
65% efficiency on A100 GPU
What Percent of Operations consist of
non-matmul?

[~~cost~~ ans $\frac{65}{100} (\text{Max Flops}) = (1 - \frac{\alpha}{100}) (\text{Max Flops})$
 $+ (\frac{\alpha}{100}) (\frac{\text{Max Flops}}{16})$
Non matmul flops are
 $\sim 16 \times$ slower on A100 GPU

$$\frac{65}{100} = 1 + \frac{\alpha}{100} \left(\frac{1}{16} - 1 \right)$$

$$\frac{\alpha}{100} \left(1 - \frac{1}{16} \right) = \frac{35}{100}$$

$$\alpha \frac{15}{16} = 35$$

$$\alpha = \frac{35 \times 16}{15} \Rightarrow \boxed{\alpha = 37.33\%}$$

$\approx 37.33\%$ are non matmul operations